# 1 Discriminative Prompt based Recalibration - DPR

After tric (bimc_ensemble_dinov2_clip) baseline and classwise prototype calculation, we recalibrate text prototypes in this step. (for each class, text prototype vector already calculated from baseline).

Lets assume, baseline text prototype vector of class-c, $T_c$
Firstly build the $S_{i,j}$, cosine similarity value of class-i prototype from class-j prototype from $T_c$

I have already created pairwise discriminative prompt matrix, M(of size n*n*k) where k is number of prompt and n is class number of the dataset.
Also, $N_{i,j}$ is new prototype vector of class i; if we pass all k prompts of class-pair (i,j) $M_{i,j,k}$ prompts in the text encoder (after passing in the baseline pipeline), and average these k values (here, k=3) to get a final class-pair wise discriminative prototype vector.
Here we have n prototypes of class-i considering dsicrimination between n classes. total n*n new prototypes for n classes. (each having n)
Now the algorithm starts with:
1. check the most similar class-pair values from all $S_{i,j}$ (n*n values total). If multiple (i,j)pairs found with same similarity, just take any one of these.
2. Now check two things to begin the recalibration steps: a) this exact pair (i, j) was not recalibrated before (one pair (i,j) gets recalibrated only once); b) the pairwise similarity value should be more than a threshold ($r_th$, this is our first hyperparameter, for now the value is 0.5. So only if inter class similarity score is more than 0.5, and these class pair were not recalibrated before, move on to next steps)
**Algorithm**:
1. if a,b classes has pairwise most similar score,

$$D_{a\to b} = \sum_{i\neq a} (S_{a,i}) * (N_{a,i})$$

$$D_{b\to a} = \sum_{i\neq b} (S_{b,i}) * (N_{b,i})$$

new prototypes vectors after classwise discriminative prompt:
$T'_a$ = a + $\alpha$ * $D_{a\to b}$
$T'_b$ = b + $\alpha$ * $D_{b\to a}$

2. After this, make a temporary pairwise similarity score vector for class a and class b by considering new prototypes, $T'_a$ and $T'_b$.
Build $tmp_{a',i}$ where $tmp_{a',i}$ is similarity score between $T'_a$ and $T_i$ that is all other classes similarity to a class's new prototype ($tmp_{a',a'}$ is redundant) similarly, $tmp_{b',i}$ for similarity of new prototype of and all other classes except b's new prototype itself.
Now the idea is to compare similarity scores of class-a and class-b with all other classes. before and after discriminative prompt. Our goal will be to reduce similarity of two closely similar classes with discriminative prompt these classes with all other classes and taking weighted avg of all these new prototype's value and add a portion of this discrimination ($\alpha$) to get new prototype; such that the similarity scores of new prototype's with all other classes is overall better than previous prototypes.

3. To check betterment, we consider two things:
a. $S_{a,b} > tmp_{a',b'}$ (The reason is we have reduced the similarity successfully between two similar classes with discriminative prototype generated from prompts)
b. If a holds then we will update $T'_a$ with $T_a$ only if

$$\left( \forall_{\substack{i \neq a \\ i \neq b}} S_{a,i} - \forall_{\substack{i \neq a' \\ i \neq b'}} tmp_{a',i} \right) > -0.1$$

That is finally check whether reducing similarity from a to b increase the similarity by 0.1 from a to any other classes or not. If it does, then new prototype is still bad as it comes with a cost, increasing the similarity with any other class(s) by a margin of 0.1 is bad for us. So no update. If similarity increases by a margin of 0.1 than before even with any one of the classes, then the update will not happen. So and operation to check the condition equation for n-2 classes (except its own class and the already calculated pair-class from 3.a. step). If all condition satisfies, then update $T'_a$ with $T_a$.
Similarly, after 3.a. step, if $\left( \forall_{\substack{i \neq a \\ i \neq b}} S_{b,i} - \forall_{\substack{i \neq a' \\ i \neq b'}} tmp_{b',i} \right) > -0.1$ only then update $T'_b$ with $T_b$. Note that, $T'_a$ and $T'_b$ update will be done independently at this last condition check of 3.b. It can be possible to update only $T'_a$ or $T'_b$ or both or none.
If any prototype update occurs, also update the similarity matrix values in $S_{a,i}$ (if a gets updated) and $S_{b,i}$ (if b gets updated) with new similarity between prototype values from $tmp_{a',i}$ for all classes except a (if a gets updated) and new similarity between prototype values from $tmp_{b',i}$ for all classes except b (if b gets updated)

After this entire algorithmic steps, go back again at step-1 to check the most similar class-pair values from all $S_{i,j}$. Then repeat this process until either every class-wise pair is checked (or visited this algorithm) at least once or none of the pair classwise similarity is greater than 0.5